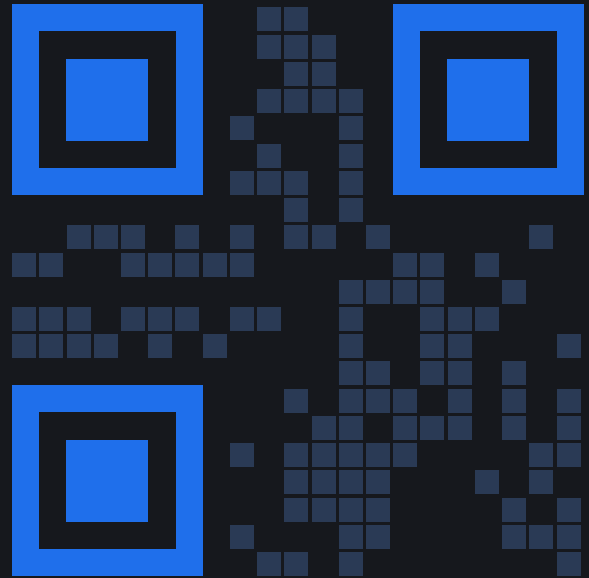




QReaper

Anti-Quishing Analyzer

Informe de la Práctica 1

Leemos el código QR antes de que lo escanee la víctima.

EQUIPO

Andrés Caso Iglesias

Jose Luis Botella

Alex Sas Muresan

Juan Francisco Jurado Naranjo

Ismael Gutiérrez

Índice

1. De qué va esto	2
2. Cómo funciona	2
Paso 1 · Leer el código QR	2
Paso 2 · Mirar la dirección sin abrirla	2
Paso 3 · Probarla en un entorno aislado	3
Paso 4 · Poner nota	3
Paso 5 · El informe	3
3. Dónde se puede usar	3
4. Requisitos de la Práctica 1	4
1.1 La aplicación web	4
1.2 La base de datos	5
1.3 La API y la herramienta MCP	5
1.4 El repositorio de GitHub	5
1.5 Del portátil a la web publicada	5
Inteligencia artificial	6
5. La seguridad de la propia herramienta	6
6. Resultados	7
7. El equipo y cómo trabajamos	7
8. Qué mejoraríamos	8
9. Para terminar	8

1. De qué va esto

Los códigos QR se han colado en el phishing. Te llega un correo con un cuadradito de puntos, lo escaneas con el móvil y acabas en una web que imita a tu banco, a Correos o a la DGT. A este tipo de fraude se le llama **quishing** (QR + phishing).

Lo que lo hace tan eficaz es que los filtros de correo no leen imágenes. Analizan texto y cabeceras, y un QR es un dibujo, así que el enlace malicioso no existe para la máquina hasta que alguien lo escanea. Para la persona tampoco es fácil: la dirección va escondida dentro del dibujo y no la ves hasta que la página ya está abierta.

QReaper ataca justo ese hueco. Le das un correo, un PDF, una imagen o una dirección web y te devuelve tres cosas: un veredicto (**PELIGRO**, **SOSPECHOSO** o **SEGURO**), una nota de riesgo de 0 a 100 y un informe que explica los motivos con palabras normales. El enlace nunca se abre en tu ordenador ni en tu móvil.

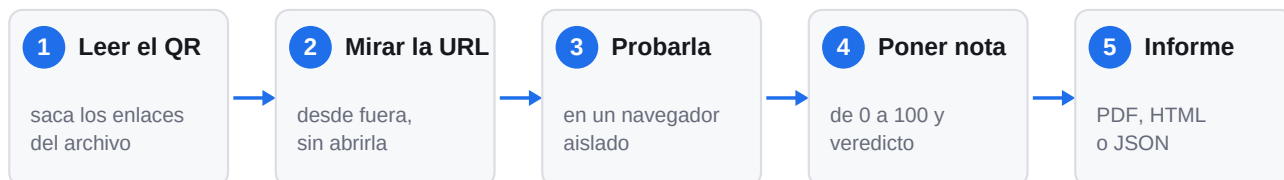
5 pasos de análisis	3 veredictos	7 endpoints de API	3 herramientas MCP	218 pruebas
-------------------------------	------------------------	------------------------------	------------------------------	-----------------------

2. Cómo funciona

Por dentro es una cadena de cinco pasos. Cada paso es un módulo distinto, lo programó una persona distinta y solo se comunica con el siguiente a través de un formato fijo que acordamos antes de escribir código. Así, si un paso falla, los demás siguen y el informe avisa de lo que ha pasado en lugar de aparentar que todo ha ido bien.

Entra: correo .eml, PDF, imagen o una dirección suelta

Salen: veredicto + nota + informe



Paso 1 · Leer el código QR

Aceptamos correos (.eml), PDF e imágenes (png, jpg, webp, bmp). Buscamos todos los QR que haya, aunque estén repetidos o partidos, y devolvemos las direcciones sin duplicados. Si la imagen es pequeña o está borrosa, la retocamos (más contraste, ampliada al doble) y lo volvemos a intentar. Si no hay ningún QR, el resultado es una lista vacía, no un error.

Paso 2 · Mirar la dirección sin abrirla

Antes de arriesgarnos a visitarla, la examinamos desde fuera. Este paso solo consulta datos públicos:

- **Antigüedad del dominio.** Una web que dice ser de Correos y se registró hace tres días huele mal.
- **Terminación sospechosa.** Hay extensiones baratas o gratuitas (.tk, .top, .xyz, .click...) donde se concentra mucho fraude.
- **Suplantación de marca.** Pilla una letra cambiada (correos.es → correrros.es), caracteres que se parecen (un 0 por una o) y marcas escondidas en el subdominio (correos.es.pago-pendiente.top). La lista de marcas está pensada para España: Correos, DGT, Agencia Tributaria, BBVA, Santander, CaixaBank, Sabadell, SEUR, DHL, Amazon, Netflix, Endesa, Iberdrola, Movistar, Vodafone y Seguridad Social.

- **Acortadores.** Si el enlace es de tipo bit.ly, averiguamos a dónde lleva de verdad.
- **Enlaces que abren otras apps** del móvil, como telegram://.

Paso 3 · Probarla en un entorno aislado

Aquí sí abrimos la dirección, pero en un navegador temporal que no es el tuyo. No tiene cookies ni sesiones (no hay nada que robar), las descargas y los permisos están bloqueados, solo admite webs normales (http y https), tiene un tiempo máximo de 20 segundos y se destruye al terminar. En el modo estricto corre dentro de un contenedor sin privilegios y con memoria y procesos limitados.

De ahí sacamos la cadena completa de redirecciones, una captura de pantalla de la página de destino y si esta pide usuario y contraseña. Además, antes de abrir nada comprobamos que la dirección no apunte a una red interna, para que un QR malicioso no pueda usar la herramienta para curiosear dentro de una organización.

Paso 4 · Poner nota

Todas las señales se juntan en una nota de 0 a 100. Cada señal suma puntos y cada punto viene con su motivo en una frase normal («la dirección imita a la de BBVA», «la página pide usuario y contraseña»). Es un sistema de reglas que se puede leer y discutir, no una caja negra. Hay combinaciones que agravan el riesgo, como imitar a una marca y pedir contraseña, o un dominio recién creado con formulario de login.

Dos detalles que nos importaban. Si el navegador aislado no consigue abrir la página, lo decimos: «no se pudo comprobar» nunca se convierte en «es seguro». Y la nota está calibrada para no dar falsas alarmas, porque un banco legítimo que pide login no puede salir como PELIGRO. Si la herramienta marca todo como fraude, la gente deja de hacerle caso.

PELIGRO	SOSPECHOSO	SEGURO
«No introduzcas ningún dato. Borra el mensaje y repórtalo.»	Trátalo como no fiable: sin credenciales ni datos de pago, y confirma por otro canal.	Sin indicios de fraude, pero mira siempre la dirección antes de escribir una contraseña.

Paso 5 · El informe

El resultado se entrega en un documento que cualquiera puede leer: **PDF** para compartir o archivar, **HTML** para verlo en el navegador y **JSON** para que otras herramientas lo aprovechen. Incluye la dirección analizada, el veredicto, las señales, la cadena de redirecciones, la captura de la página, el desglose de la nota y una recomendación de qué hacer. El color cambia con el veredicto (rojo, ámbar, verde), pero el texto siempre lo acompaña, para que nadie dependa solo del color.

Un detalle de diseño: el botón que abre el enlace analizado solo aparece cuando el veredicto es SEGURO. La herramienta nunca te lleva por su cuenta a una dirección sospechosa.

3. Dónde se puede usar

El motor es uno solo y todas las puertas de entrada hablan con él. Por eso nadie recibe dos respuestas distintas para el mismo enlace. La idea era llevar la herramienta al sitio donde la gente se encuentra el fraude, en vez de pedirle que cambie sus hábitos.

Canal	Para quién	Qué permite
Web (qreaper.es)	Cualquiera, sin instalar nada	Escanear un QR con la cámara, subir o arrastrar un archivo, o pegar una dirección
App instalable (PWA)	Móvil	Se instala como una app y trae accesos directos, por ejemplo para escanear un QR
Menú «Compartir»	Móvil	Compartir un enlace o una imagen desde cualquier app y ver el veredicto
Extensión de navegador	Ordenador	Chrome, Edge y Firefox. Analiza los enlaces de un correo abierto sin copiar y pegar
Gmail y Outlook	Correo	Analizar el mensaje que estás leyendo sin salir del correo
Línea de comandos	Perfiles técnicos	qreaper analizar correo.eml, para scripts y automatizaciones
API	Otras aplicaciones	Documentación interactiva en qreaper.es/docs
MCP	Asistentes de IA	Pedirle a Claude «analiza este enlace sospechoso» y que use QReaper por su cuenta

En qreaper.es/instalar está la guía para tenerlo a mano en el móvil, el correo y el navegador.

4. Requisitos de la Práctica 1

La práctica pedía cinco cosas obligatorias y dejaba la inteligencia artificial como opcional. Primero un resumen y luego cada punto con más detalle.

Requisito	Qué hicimos	Estado
1.1 Aplicación web operativa	Web pública con HTTPS en qreaper.es, en un servidor propio con IP pública	Cumplido
1.2 Base de datos real	SQLite con el historial de análisis, persistente en un volumen de Docker	Cumplido
1.3 API o webhook real	API REST con 7 endpoints y documentación Swagger, más un servidor MCP con 3 herramientas para usarlo desde Claude	Cumplido
1.4 Repositorio en GitHub con README	Repositorio principal y repositorio público del MCP, con README de instalación, uso y despliegue	Cumplido
1.5 Comunicación y arquitectura	Flujo portátil → GitHub → servidor, con SSH por par de claves y publicación por HTTPS	Cumplido
IA y ML (opcional)	Sin aprendizaje automático: reglas explicables. La IA entra por el MCP	Coherente

1.1 La aplicación web

La web está en <https://qreaper.es/>, con el candado de HTTPS y también accesible desde www.qreaper.es y qreaper.b-dev.es. Desde ahí se puede escanear un QR con la cámara, subir o arrastrar un archivo y pegar una dirección. Está en un servidor propio, no en un túnel ni en un equipo local, así que cualquiera puede probarla desde fuera.

1.2 La base de datos

Usamos **SQLite**. Cada análisis que se hace queda guardado con la fecha, el archivo, la dirección, el veredicto, la nota y los motivos, y nada más. La aplicación escribe en la base de datos con cada análisis y la lee cuando se consulta el historial, tanto por API como desde Claude. Elegimos SQLite por sencillez y porque para este volumen de datos va de sobra. Vive en un volumen de Docker, así que no se pierde cuando actualizamos la aplicación.

1.3 La API y la herramienta MCP

No quisimos un único endpoint que responde «200 OK». La API tiene siete rutas con acciones distintas y su documentación interactiva se genera sola en **greaper.es/docs**, de modo que se puede recorrer entera desde el navegador.

Método	Ruta	Qué hace
GET	/	Datos básicos y lista de endpoints
GET	/salud	Comprueba que el servicio está funcionando
POST	/analizar/url	Analiza una dirección y devuelve veredicto, nota y motivos
POST	/analizar/archivo	Recibe una imagen, un PDF o un correo, extrae los QR y analiza cada enlace
POST	/analizar/lote	Analiza varias direcciones a la vez (sin duplicados y con un máximo de 15)
GET	/historial	Lista los análisis guardados, filtrables por veredicto y cantidad
GET	/historial/{id}	Devuelve un análisis concreto

Encima de la misma lógica montamos el **servidor MCP**, que es lo que permite llamar a QReaper desde Claude Desktop o Claude Code. Se registra una sola vez con `claude mcp add greaper -- greaper-mcp` y desde ahí se le habla en lenguaje normal: «analiza esta URL sospechosa», «escanea esta imagen» o «enséñame las últimas 10 detecciones». Tiene tres herramientas: `analizar_url`, `analizar_archivo` e `historial`. Las respuestas son un resumen corto (veredicto, nota, motivos y dirección final) para no malgastar tokens del asistente.

1.4 El repositorio de GitHub

El código está en GitHub, en **bdjoseluis/greaper** (repositorio principal) y **bdjoseluis/greaper-mcp** (público, con la herramienta MCP). El README explica qué es el proyecto, cómo se instala, cómo se usa (línea de comandos, API y MCP), cómo se despliega y cómo está organizado el código. Hay también una carpeta de documentación (API, despliegue, organización del equipo y contratos entre módulos). Cada persona trabajó en su rama y todo entró por pull request: unos 64 commits en el repositorio principal.

1.5 Del portátil a la web publicada

Este es el recorrido completo, desde que alguien escribe una línea hasta que se ve en internet:



SSH solo con clave pública, sin contraseña. Puerto filtrado con `ufw` y `fail2ban`.
El contenedor no abre puertos al exterior: la única puerta pública es Traefik, con HTTPS.

- **Local.** Programamos en VS Code o Claude Code y subimos los cambios a GitHub con `git push`.
- **GitHub.** Es el almacén central. Los cambios entran por pull request y los revisa Jose Luis.
- **Servidor.** Un VPS de Hetzner, en Alemania, con IP pública. Nos conectamos por SSH con un par de claves: la privada se queda en el portátil y la pública está en el servidor, que no admite contraseña. El puerto SSH está filtrado con `ufw` y `fail2ban`.
- **Publicación.** La aplicación corre en un contenedor Docker que escucha en el puerto 8000 y no se expone directamente. Delante hay Traefik, que se encarga del HTTPS con certificado de Let's Encrypt (se renueva solo) y reenvía el tráfico al contenedor.
- **Acceso.** El dominio apunta a la IP del servidor y la web se abre desde cualquier sitio.

Para actualizar, en el servidor basta con esto:

```
cd /opt/qreaper && git pull
docker compose -f deploy/docker-compose.vps.yml up -d --build
curl -s https://qreaper.es/salud
```

El contenedor tiene un límite de 400 MB de memoria y los registros rotan solos. Lo más delicado, que es abrir enlaces hostiles en el navegador aislado, se enciende o se apaga con una variable de configuración (`QREAPER_SANDBOX`) sin tocar el código. Solo se activa donde el entorno está preparado para ello.

Inteligencia artificial

No usamos aprendizaje automático y tampoco lo vendemos como tal. El veredicto sale de reglas que se pueden leer, y eso nos parecía una ventaja: cada punto de la nota tiene una razón que se puede discutir. La IA entra por otro lado: el servidor MCP deja que Claude use QReaper, y las respuestas compactas ahorran tokens.

5. La seguridad de la propia herramienta

Una herramienta de seguridad que se deja hackear no sirve de mucho, así que le dedicamos tiempo a esto:

- **Navegador desechable.** Cada análisis usa un perfil nuevo, sin cookies ni sesiones. Aunque la web sea maliciosa, no hay credenciales que robar.
- **Contenedor sin privilegios.** En modo estricto, la prueba corre sin permisos especiales, con memoria y procesos limitados, y el contenedor se elimina al terminar.
- **Red interna protegida.** Se bloquean las direcciones privadas y las de servicios internos de la nube, para que un QR no sirva de trampolín hacia dentro de una organización.
- **Subidas controladas.** Solo se aceptan formatos permitidos, con límite de tamaño, y se comprueba que el archivo es lo que dice ser (cambiarle la extensión no basta). Las imágenes tienen un máximo de dimensiones.
- **HTTPS y cabeceras de seguridad** en el servicio público.
- **Datos mínimos.** Solo se guarda lo necesario para el historial, y hay una página de privacidad.
- **Pruebas inocuas.** Los dominios fraudulentos del banco de pruebas son inventados. No hay ningún enlace de phishing real en el repositorio.

6. Resultados

Generamos informes reales sobre varias direcciones. Estos son tres ejemplos representativos:

Dirección analizada	Veredicto	Nota
https://b-dev.es/ sitio legítimo	SEGURO	5 / 100
http://bbva-verify.tk/login suplanta a BBVA	PELIGRO	75 / 100
http://dgt-multas-online.tk/consulta suplanta a la DGT	SOSPECHOSO	45 / 100

Para medir los aciertos y también las falsas alarmas montamos un banco de pruebas con **60 códigos QR**: 30 legítimos y 30 fraudulentos simulados, emparejados uno a uno, de modo que cada caso legítimo tiene su gemelo fraudulento. Cualquier ajuste de la nota tiene que volver a pasar por ahí antes de publicarse.

Indicador	Valor	Indicador	Valor
Módulos de programa	16	Pruebas automatizadas	218
Líneas de código Python	unas 4.900	QR del banco de pruebas	60
Commits (repo principal)	64	Documentos en docs/	12
Semanas de desarrollo	7	Rutas web del servicio	27

7. El equipo y cómo trabajamos

Integrante	Perfil	Qué hizo
Jose Luis Botella	Ingeniería de software	Núcleo: navegador aislado, puntuación, orquestador, API y MCP. Revisión de los pull requests
Andrés Caso Iglesias	Programación (DAM)	Lectura y descifrado de los códigos QR
Alex Sas Muresan	Telecomunicaciones	Análisis estático de direcciones y dominios
Juan Francisco Jurado Naranjo	Telecomunicaciones	Informes y banco de pruebas
Ismael	Redes y telecomunicaciones	Línea de comandos, interfaz y documentación. Dominio qreaper.es

Antes de escribir una línea de código definimos, en un documento de **contratos**, qué recibe y qué devuelve cada módulo. Funcionó como un pliego de condiciones interno: cada uno pudo programar el suyo sin esperar a los demás y sin romper lo de nadie. Es la razón principal de que cinco personas trabajáramos en paralelo durante siete semanas sin bloquearnos.

Trabajamos en cuatro sprints, y lo que vino después fue el despliegue y los canales de acceso:

Sprint	Fechas	Objetivo
0 · Puesta a punto	5 – 10 ago	Repositorio, entorno y una primera prueba por persona
1 · Módulos	11 – 24 ago	Cada módulo funciona solo y pasa sus pruebas
2 · Integración	25 ago – 1 sep	Todo encaja en la cadena; banco de pruebas e informes reales
3 · Pulido	2 – 5 sep	Demo, documentación y últimos ajustes

Las normas eran sencillas: nadie escribe directamente en la rama principal, todo entra por pull request y una tarea solo se da por hecha cuando funciona, su prueba pasa y la integración está revisada. Para organizarnos usamos un tablero con 54 tarjetas agrupadas por sprint. Las 218 pruebas se pueden lanzar de una vez para comprobar que un cambio no ha roto nada. Cubren cada módulo por separado y el sistema completo, y una de ellas vigila que el informe nunca se invente datos cuando falta información.

8. Qué mejoraríamos

Lo que hay funciona de punta a punta, pero tenemos claro por dónde seguiríamos:

- **Afinar la nota con más casos reales** (anonimizados). La estructura es correcta, pero con más campañas se ajustarían mejor los umbrales entre veredictos.
- **Consultar listas públicas de dominios fraudulentos** para tener una segunda opinión además de nuestras señales.
- **Guardar los resultados en una caché**. Hoy, analizar dos veces la misma dirección repite todo el trabajo.
- **Integración continua** que lance las 218 pruebas en cada cambio.
- **Probar la accesibilidad con lector de pantalla**. Ya evitamos depender solo del color, pero no lo hemos validado con herramientas de apoyo.

9. Para terminar

El quishing funciona porque nadie mira lo que hay dentro del cuadradito. QReaper lo mira por ti, lo prueba donde no puede hacer daño y te lo cuenta en cristiano. Lo hemos hecho de modo que se pueda usar desde donde llega el fraude (el móvil, el correo, el navegador), y que cada decisión se pueda explicar y revisar.

En una frase: QReaper convierte una imagen que nadie puede leer en una decisión explicada.

Enlace	Dónde
Web del proyecto	https://qreaper.es/
Documentación de la API	https://qreaper.es/docs
Guía de instalación (móvil, correo y navegador)	https://qreaper.es/instalar
Repositorio principal	https://github.com/bdjoseluis/qreaper
Repositorio MCP (público)	https://github.com/bdjoseluis/qreaper-mcp
Instagram	https://www.instagram.com/qreaper.es/